Author: Micheal Lanham

SECTION 1

Strategies for Consuming Agents

- **Embedded web and voice:** Browser-side agents using the Realtime SDK suit low-latency voice UX when tools are light; production must mint ephemeral keys server-side rather than shipping provider API keys to clients.
- **Browser constraints:** Client-side deployment faces key exposure, CORS limits, and shared rate quotas; embedding works when a backend already proxies credentials and heavy tools.
- **API microservice:** Wrapping an agent in FastAPI or similar fits request or response work such as image generation, lookups, and summarization with typed POST bodies and long-running responses.
- **Voice plus backend tool:** A realtime browser agent can call a remote generate endpoint as a tool, keeping conversation live while slow work runs asynchronously behind queues and workers in full production stacks.
- **Pattern selection:** Consumption style—inline app, hosted service, or tool for other agents over MCP or A2A—typically drives how the agent is packaged and scaled.

SECTION 2

Dockerizing Agent Systems

- **Microservice fit:** Agents are self-contained concerns ideal for containers: swap versions, scale workers independently, and isolate dependencies.
- **Single-service container:** A Dockerfile standardizes Python runtime, installs requirements, exposes a port, and runs the API with injected secrets at runtime—not baked into the image.
- **Docker Compose:** Compose declares the full multi-agent stack—web UI, image worker, search worker—with shared environment variables and dependency ordering for one-command local orchestration.
- **Local operations:** Docker Desktop supports log inspection, restart, and named containers for iterative development before cloud deployment.
- **Tunneling caveat:** Tools like localtunnel help demos and webhook debugging but are not production substitutes because public URLs bypass normal network and auth boundaries.

SECTION 3

Advanced Deployment Strategies

- **Runtime choice:** Edge for conversational latency, synchronous API for standard request or response, and event-driven workers for long or bursty jobs that exceed HTTP timeouts.
- **Three wires:** WebRTC or WebSocket for full-duplex voice; HTTP plus SSE for streamed tool and MCP calls; message buses for decoupled background execution while the session continues.
- **Front-door topology:** A lightweight front-door agent routes intent to specialized worker agents that may each use different latency-appropriate communication patterns.
- **State and idempotency:** Short-term chat in fast stores, long-term knowledge via shared vector indexes; idempotent tools keyed by canonical inputs enable cache, replay, and safe retries.
- **Release engineering:** Version prompts, tools, and models; promote through offline tests, shadow traffic, canary, and rollback; pin exact model and tool endpoints per turn for reproducibility.
- **Observability, reliability, cost:** Trace UI through gateway, agent, tools, and model with session and turn correlation; enforce timeouts, fallbacks, and circuit breakers; control spend via context trimming, caching, and intent-based model routing measured against value delivered.

SECTION 4

Security, Safety, and Governance

- **Threat model:** Map assets—keys, tool credentials, data read and written, logs, PII—to surfaces including client, gateway, runtime, tool servers, model provider, and storage.
- **Prompt injection:** Direct and indirect injection via retrieved content require layered defenses: treat tool output as data, hierarchy of instructions, output filtering, and human gates on irreversible actions.
- **Identity and secrets:** Agents act with user-scoped authorization; mint short-lived realtime secrets after auth; inject secrets via environment or secret managers with least privilege and rotation.
- **Tool safety:** Sandbox execution, restricted filesystem paths, outbound allowlists, and resource caps; understand MCP server trust and enforce HTTPS plus auth on external tools.
- **Policy enforcement:** Content safety, privacy compliance, audit logs, rate limits, access control, and human-in-the-loop for high-stakes actions belong outside fragile prompt-only enforcement, often via a policy registry with logged overrides.